Spring Cloud Function provides a `@FunctionalSpringBootTest` annotation you can use to set up the context for your integration tests. Unlike unit tests, you don't want to invoke the function directly but rather ask the framework to provide that for you. All the functions managed by the framework are available through the `FunctionCatalog`, an object that acts as a function registry. When the framework serves the function, it doesn't only contain the implementation you wrote; it's enhanced with extra features offered by Spring Cloud Function, like transparent type conversion and function composition. Let's see how this works.

First, you need to add a test dependency on Reactor Test in the build.gradle file, since part of the business logic is implemented using Reactor. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 10.10 Adding dependency for Reactor Test in Dispatcher Service

```
dependencies {
  ...
  testImplementation 'io.projectreactor:reactor-test'
}
```

Then, in the src/test/java folder of your Dispatcher Service project, create a new `DispatchingFunctionsIntegrationTests` class. You can write integration tests for the two functions individually, but it's more interesting verifying the behavior of the composed function, `pack()` + `label()`, as provided by Spring Cloud Function.

Listing 10.11 Integration tests for a function composition

```
package com.polarbookshop.dispatcherservice;

import java.util.function.Function;
import org.junit.jupiter.api.Test;
import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
```